LaneDetection

December 18, 2025

1 BITS Pilani - Computer Vision

2 Straight Lane Detection in Road Images

2.1 Assignment 1: Straight Lane Detection in Road Images

Group Members:

Name	ID	Contribution
Ashvini Satish Bhosale	2024aa05732	100%
Rajesh Talari	2024aa05071	100%
Pramod TK	2024ab05128	100%
Rahul Agnihotri	2024aa05347	100%
Pagi Sandeep Satish Pratibha	2024aa05545	100%

2.2 Import Required Libraries

[1]:
```
# ============================================================================
# SECTION 1: IMPORT REQUIRED LIBRARIES AND MODULES
# ============================================================================

import cv2                                  # Image processing, edge detection, Hough
import numpy as np                          # Numerical operations
import matplotlib.pyplot as plt             # Visualization
import matplotlib.patches as patches        # Drawing patches
from matplotlib.patches import Polygon      # Polygon drawing
from sklearn.preprocessing import StandardScaler  # Feature scaling
from sklearn.cluster import KMeans          # K-Means clustering
import json                                 # JSON reporting
import time                                 # Performance timing
from pathlib import Path                    # File path handling
from collections import deque              # Fast queue operations
import warnings                             # Warning suppression
```

2.3 Preprocessing and Edge Detection

2.3.1 Data Acquisition : Load and structure the dataset

[2]:
```
# ================================================================================
# SECTION 2: DATA ACQUISITION
# ================================================================================
class LaneDetectionDataSetLoader():
    """Class to load and manage lane detection dataset."""
    def __init__(self, dataset_path):
        self.dataset_path = Path(dataset_path)
        self.images = []
        self.image_paths = []
        self.stats = {
            'total_images': 0,
            'image_format': 'PNG/JPG',
            'image_dimensions': None
        }

    def load_images(self, max_images=None):
        """Load images from the dataset path."""
        image_files = list(self.dataset_path.glob("*.jpg"))
        if max_images:
            image_files = image_files[:max_images]
        image_files = sorted(set(image_files))[:max_images]
        for img_file_path in image_files:
            img = cv2.imread(str(img_file_path))
            if img is not None:
                self.images.append(img)
                self.image_paths.append(img_file_path)

        print(f"Loaded {len(self.images)} images from {self.dataset_path}")

path = "/Users/pramodtk/Documents/MTech/Semister 3/Computer Vision/CodeSamples/
 ↪Learn_CV_with_Dhruv-master/Assignment-1/kaggle-images/road_line_images"
images_loader = LaneDetectionDataSetLoader(path)
images_loader.load_images(max_images=20)
print (f"Type images loaded: {type(images_loader.images)}")
```

```
Loaded 20 images from /Users/pramodtk/Documents/MTech/Semister 3/Computer
Vision/CodeSamples/Learn_CV_with_Dhruv-master/Assignment-1/kaggle-
images/road_line_images
Type images loaded: <class 'list'>
```

2.3.2 Convert it to grayscale and apply a Gaussian Blur to reduce noise

[3]:
```
# ============================================================================
# SECTION 3: DATA PREPARATION
# ============================================================================
class ImagePreprocessor:
    """
    Handles image preprocessing including:
    - Resizing
    - Grayscale conversion
    - Color space conversion (HLS, HSV)
    - Gaussian blurring
    - Histogram equalization

    Rubric: Resize, convert to grayscale, apply histogram equalization,
            Gaussian blur, document preprocessing choices
    """

    def __init__(self, target_height=480, target_width=640):
        """Initialize preprocessor with target dimensions."""
        self.target_height = target_height
        self.target_width = target_width

    def to_grayscale(self, image):
        """Convert image to grayscale."""
        return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)


    def apply_gaussian_blur(self, image, kernel_size=(5, 5), sigma=0):
        """Apply Gaussian Blur to the image."""
        return cv2.GaussianBlur(image, kernel_size, sigma)

    def preprocess(self, image, enhance_lines=True):
        """Complete preprocessing pipeline."""
        # Step 1: Resize
        resized = cv2.resize(image, (self.target_width, self.target_height))

        # Step 2: Convert BGR to RGB for visualization
        original_rgb = cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)

        # Step 3: Convert to different color spaces for analysis
        if enhance_lines:
            # HLS color space: Better for white/yellow line detection
            hls = cv2.cvtColor(resized, cv2.COLOR_BGR2HLS)

            l_channel = hls[:, :, 1]  # Lightness channel
            s_channel = hls[:, :, 2]  # Saturation channel
```

```
            # Enhance white and yellow lines
            # White lines: High lightness, low saturation
            white_mask = (l_channel > 180) & (s_channel < 50)


            # Yellow lines: Specific hue range (in HSV)
            hsv = cv2.cvtColor(resized, cv2.COLOR_BGR2HSV)
            lower_yellow = np.array([15, 100, 100])
            upper_yellow = np.array([35, 255, 255])
            yellow_mask = cv2.inRange(hsv, lower_yellow, upper_yellow)

            # Combine masks
            line_mask = cv2.bitwise_or(
                white_mask.astype(np.uint8) * 255,
                yellow_mask
            )

            # Grayscale conversion
            gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)

            # Enhance using line mask
            processed = cv2.bitwise_or(gray, line_mask)
        else:
            gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)
            processed = gray

        # Step 4: Histogram Equalization (enhance contrast)
        processed = cv2.equalizeHist(processed)

        # Step 5: Gaussian Blur (reduce noise)
        # Kernel size: 5x5 (good balance between noise reduction and detail␣
↪preservation)
        processed = cv2.GaussianBlur(processed, (5, 5), 1.0)

        return processed, original_rgb
```

[4]:
```
# ============================================================================
# SECTION 4: FEATURE ENGINEERING
# ============================================================================
class FeatureExtractor:
    """
    Extracts lane line features using:
    - Canny Edge Detection
    - Region of Interest (ROI) masking
    - Hough Transform for line detection
```

```
    """

    def __init__(self, canny_low=50, canny_high=150, hough_threshold=20):
        """
        Initialize feature extractor.

        Args:
            canny_low: Low threshold for Canny edge detection
            canny_high: High threshold for Canny edge detection
            hough_threshold: Minimum votes in Hough accumulator
        """
        self.canny_low = canny_low
        self.canny_high = canny_high
        self.hough_threshold = hough_threshold

        print("[4] FEATURE ENGINEERING")
        print("-" * 80)
        print(f"   Canny Edge Detection:")
        print(f"     - Low Threshold (low): {canny_low}")
        print(f"     - High Threshold (high): {canny_high}")
        print(f"     - Ratio: {canny_high/canny_low:.1f}:1 (optimal)")
        print(f"   Hough Transform:")
        print(f"     - Method: Probabilistic Hough Line Transform")
        print(f"     - Threshold (min votes): {hough_threshold}")
        print()

    def detect_edges(self, processed_image):
        """
        Apply Canny edge detection.

        Mathematical basis:
        1. Compute image gradients: ∇I = [∂I/∂x, ∂I/∂y]
        2. Non-maximum suppression: Thin edges
        3. Double thresholding: Classify pixels
        4. Edge tracking by hysteresis: Connect weak edges
        """
        edges = cv2.Canny(
            processed_image,
            self.canny_low,
            self.canny_high
        )
        return edges

    def create_roi_mask(self, image_shape):
        """
        Create Region of Interest (ROI) mask.
```

```
        ROI is defined as a trapezoid:
        - Covers the area in front of the vehicle
        - Excludes sky and irrelevant areas
        """
        height, width = image_shape[:2]

        # Define trapezoid vertices
        vertices = np.array([
            [0, height],                    # Bottom left
            [width * 0.35, height * 0.55],  # Upper left
            [width * 0.65, height * 0.55],  # Upper right
            [width, height]                 # Bottom right
        ], dtype=np.int32)

        # Create mask
        mask = np.zeros(image_shape[:2], dtype=np.uint8)
        cv2.fillPoly(mask, [vertices], 255)

        return mask

    def apply_roi(self, edges, roi_mask):
        """Apply ROI mask to edge image."""
        masked_edges = cv2.bitwise_and(edges, roi_mask)
        return masked_edges

    def detect_hough_lines(self, masked_edges):
        """
        Apply Probabilistic Hough Transform to detect lines.

        Mathematical basis:
        In image space: Line defined as y = mx + b
        In Hough space: Line mapped to point ( ,  )

        Transformation:   = x·cos( ) + y·sin( )
        """
        lines = cv2.HoughLinesP(
            masked_edges,
            rho=1,
            theta=np.pi / 180,
            threshold=self.hough_threshold,
            minLineLength=30,
            maxLineGap=5
        )

        return lines if lines is not None else np.array([])
```

```
# Initialize feature extractor
extractor = FeatureExtractor(canny_low=50, canny_high=150, hough_threshold=18)
print("    Feature extractor initialized\n")
```

```
[4] FEATURE ENGINEERING
--------------------------------------------------------------------------------
   Canny Edge Detection:
     - Low Threshold ( low): 50
     - High Threshold ( high): 150
     - Ratio: 3.0:1 (optimal)
   Hough Transform:
     - Method: Probabilistic Hough Line Transform
     - Threshold (min votes): 18

    Feature extractor initialized
```

[5]:
```
# ================================================================================
# SECTION 5: MODEL BUILDING
# ================================================================================

class LineFitter:
    """
    Machine Learning-based line fitting and clustering.

    Implements three methods:
    1. Simple Averaging: Mean slope and intercept
    2. RANSAC: Robust fitting with outlier rejection
    3. K-Means: Clustering in (slope, intercept) space
    """

    def __init__(self, method='averaging', slope_threshold=0.3):
        """
        Initialize line fitter.

        Args:
            method: 'averaging', 'ransac', or 'kmeans'
            slope_threshold: Min slope magnitude to filter horizontal lines
        """
        self.method = method
        self.slope_threshold = slope_threshold
        self.left_line = None
        self.right_line = None

    def extract_line_parameters(self, lines):
        """
        Extract slope and intercept from line segments.
```

```
        For line segment (x1,y1,x2,y2):
        - slope m = (y2-y1)/(x2-x1)
        - intercept b = y1 - m*x1
        """
        line_params = []

        if len(lines) == 0:
            return line_params

        for line in lines:
            x1, y1, x2, y2 = line[0]

            if x2 - x1 == 0:
                continue

            slope = (y2 - y1) / (x2 - x1)

            if abs(slope) < self.slope_threshold:
                continue

            intercept = y1 - slope * x1

            line_params.append({
                'slope': slope,
                'intercept': intercept,
                'x1': x1, 'y1': y1, 'x2': x2, 'y2': y2
            })

        return line_params

    def classify_lines(self, line_params):
        """
        Classify lines into left and right lanes by slope sign.

        Left Lane: slope < -  (negative slope)
        Right Lane: slope > +  (positive slope)
        Noise: |slope|    (near-horizontal, discarded)
        """
        left_lines = [lp for lp in line_params if lp['slope'] < -self.
↪slope_threshold]
        right_lines = [lp for lp in line_params if lp['slope'] > self.
↪slope_threshold]
        return left_lines, right_lines

    def fit_averaging(self, lines):
        """
```

```
        Simple averaging method: mean slope and intercept.

        Mathematical Definition:
        m̄ = (1/n) · Σ(m_i)
        b̄ = (1/n) · Σ(b_i)

        Pros: Fast, simple
        Cons: Sensitive to outliers
        """
        if len(lines) == 0:
            return None

        slopes = [line['slope'] for line in lines]
        intercepts = [line['intercept'] for line in lines]

        return {
            'slope': np.mean(slopes),
            'intercept': np.mean(intercepts),
            'method': 'averaging',
            'num_samples': len(lines)
        }

    def fit_ransac(self, lines, iterations=100, threshold=10):
        """
        RANSAC (Random Sample Consensus) for robust fitting.

        Algorithm:
        1. Randomly select sample → fit line
        2. Count inliers (points close to line)
        3. Repeat iterations times
        4. Return fit with maximum inliers
        """
        if len(lines) < 2:
            return self.fit_averaging(lines) if lines else None

        features = np.array([[line['slope'], line['intercept']] for line in␣
↪lines])
        best_fit = None
        max_inliers = 0

        for _ in range(iterations):
            idx = np.random.randint(0, len(features))
            sample = features[idx]
            distances = np.abs(features - sample).sum(axis=1)
            inliers = np.sum(distances < threshold)

            if inliers > max_inliers:
```

```
                max_inliers = inliers
                best_fit = {
                    'slope': sample[0],
                    'intercept': sample[1],
                    'method': 'ransac',
                    'num_samples': len(lines),
                    'inliers': max_inliers
                }

        return best_fit if best_fit else self.fit_averaging(lines)

    def fit_kmeans(self, lines, k=1):
        """
        K-Means clustering in (slope, intercept) space.

        Approach: Cluster line parameters, use cluster center as final fit
        """
        if len(lines) == 0:
            return None

        features = np.array([[line['slope'], line['intercept']] for line in␣
↪lines])

        if len(features) == 1:
            return {
                'slope': features[0, 0],
                'intercept': features[0, 1],
                'method': 'kmeans',
                'num_samples': len(lines)
            }

        scaler = StandardScaler()
        features_scaled = scaler.fit_transform(features)

        kmeans = KMeans(n_clusters=min(k, len(features)), random_state=42,␣
↪n_init=10)
        kmeans.fit(features_scaled)

        # center = scaler.inverse_transform(kmeans.cluster_centers_[0])
        center = scaler.inverse_transform(kmeans.cluster_centers_[0].reshape(1,␣
↪-1))[0]

        return {
            'slope': center[0],
            'intercept': center[1],
            'method': 'kmeans',
            'num_samples': len(lines),
```

```
            'inertia': kmeans.inertia_
        }

    def fit_lines(self, line_params):
        """
        Main fitting method using selected algorithm.
        """
        left_lines, right_lines = self.classify_lines(line_params)

        if self.method == 'averaging':
            left_fit = self.fit_averaging(left_lines)
            right_fit = self.fit_averaging(right_lines)
        elif self.method == 'ransac':
            left_fit = self.fit_ransac(left_lines)
            right_fit = self.fit_ransac(right_lines)
        elif self.method == 'kmeans':
            left_fit = self.fit_kmeans(left_lines, k=1)
            right_fit = self.fit_kmeans(right_lines, k=1)
        else:
            left_fit = self.fit_averaging(left_lines)
            right_fit = self.fit_averaging(right_lines)

        self.left_line = left_fit
        self.right_line = right_fit

        return left_fit, right_fit


print("[5] MODEL BUILDING - LINE FITTING")
print("-" * 80)
print("   Three ML-based fitting methods:")
print("     1. Averaging: O(n), simple, good for clean data")
print("     2. RANSAC: Robust to ~50% outliers")
print("     3. K-Means: Feature space clustering")
print("   Recommended: AVERAGING (fast + robust pipeline)\n")
```

```
[5] MODEL BUILDING - LINE FITTING
--------------------------------------------------------------------------------
   Three ML-based fitting methods:
     1. Averaging: O(n), simple, good for clean data
     2. RANSAC: Robust to ~50% outliers
     3. K-Means: Feature space clustering
   Recommended: AVERAGING (fast + robust pipeline)
```

[6]:
```
# ================================================================================
# COMPLETE PIPELINE
```

```python
# ============================================================================
class LaneDetectionPipeline:
    """
    Complete lane detection pipeline combining all stages.
    """

    def __init__(self,
                 canny_low=50,
                 canny_high=150,
                 hough_threshold=20,
                 fitting_method='averaging'):
        """Initialize the complete pipeline."""
        self.preprocessor = ImagePreprocessor()
        self.extractor = FeatureExtractor(canny_low, canny_high,␣
↪hough_threshold)
        self.fitter = LineFitter(method=fitting_method)

    def process_image(self, image):
        """
        Process a single image through the complete pipeline.
        """
        print("[Pipeline] Starting image processing...")

        # Step 1: Preprocessing
        processed, original_rgb = self.preprocessor.preprocess(image)
        # print(f"[Pipeline] Preprocessing done.")

        # Step 2: Edge Detection
        edges = self.extractor.detect_edges(processed)
        # print(f"[Pipeline] Edge detection done.")

        # Step 3: ROI Masking
        roi_mask = self.extractor.create_roi_mask(edges.shape)
        masked_edges = self.extractor.apply_roi(edges, roi_mask)
        # print(f"[Pipeline] ROI masking done.")

        # Step 4: Line Detection
        lines = self.extractor.detect_hough_lines(masked_edges)
        # print(f"[Pipeline] Line detection done.")

        # Step 5: Feature Extraction
        line_params = self.fitter.extract_line_parameters(lines)
        # print(f"[Pipeline] Feature extraction done.")

        # Step 6: Line Fitting
        left_fit, right_fit = self.fitter.fit_lines(line_params)
        print(f"[Pipeline] Line fitting done.")
```

```
        return {
            'original': original_rgb,
            'processed': processed,
            'edges': edges,
            'roi_mask': roi_mask,
            'masked_edges': masked_edges,
            'lines': lines,
            'line_params': line_params,
            'left_fit': left_fit,
            'right_fit': right_fit
        }

    def draw_lanes(self, image, left_fit, right_fit):
        """
        Draw detected lane lines on the original image.
        """
        annotated = image.copy()
        height, width = image.shape[:2]

        # Draw left lane
        if left_fit:
            y_points = np.array([0, height])
            x_points = (y_points - left_fit['intercept']) / left_fit['slope']
            x_points = np.clip(x_points, 0, width).astype(int)
            cv2.line(annotated,
                     (x_points[0], y_points[0]),
                     (x_points[1], y_points[1]),
                     (0, 255, 0), 3)  # Green

        # Draw right lane
        if right_fit:
            y_points = np.array([0, height])
            x_points = (y_points - right_fit['intercept']) / right_fit['slope']
            x_points = np.clip(x_points, 0, width).astype(int)
            cv2.line(annotated,
                     (x_points[0], y_points[0]),
                     (x_points[1], y_points[1]),
                     (0, 255, 0), 3)  # Green

        return annotated


# Initialize pipeline for averaging, ransac or kmeans fitting
pipelines = []
pipelines.append(LaneDetectionPipeline(
    canny_low=45,          # ← Lower (catch fainter edges)
    canny_high=145,        # ← Keep 3:1 ratio optimal
```

```
    hough_threshold=18,    # ← Lower (catch shorter segments)
    fitting_method='averaging'
))

# pipelines.append(LaneDetectionPipeline(
#     canny_low=50,
#     canny_high=150,
#     hough_threshold=20,
#     fitting_method='ransac'
# ))

# pipelines.append(LaneDetectionPipeline(
#     canny_low=50,
#     canny_high=150,
#     hough_threshold=20,
#     fitting_method='kmeans'
# )  )

print("  Pipelines initialized successfully\n")
```

```
[4] FEATURE ENGINEERING
--------------------------------------------------------------------------------
   Canny Edge Detection:
     - Low Threshold ( low): 45
     - High Threshold ( high): 145
     - Ratio: 3.2:1 (optimal)
   Hough Transform:
     - Method: Probabilistic Hough Line Transform
     - Threshold (min votes): 18

 Pipelines initialized successfully
```

[7]:
```
# ================================================================================
# SECTION 6 & 7: VALIDATION & INFERENCE
# ================================================================================

print("\n[6-7] VALIDATION & INFERENCE")
print("-" * 80)

# Process test images
test_indices = list(range(min(30, len(images_loader.images))))
pipeline_results = {}
for pipeline in pipelines:
    results = []
    for idx in test_indices:
        print(f"\n   Processing Image {idx + 1}/{len(test_indices)}")
```

```
        image = images_loader.images[idx]
        result = pipeline.process_image(image)
        results.append(result)
        print("    Image processed")
        print("result", type(result))
        if result['left_fit']:
            m, b = result['left_fit']['slope'], result['left_fit']['intercept']
            print(f"    Left Lane:   DETECTED (m={m:.3f}, b={b:.1f})")
        else:
            print(f"    Left Lane:   NOT DETECTED")

        if result['right_fit']:
            m, b = result['right_fit']['slope'],␣
 ↪result['right_fit']['intercept']
            print(f"    Right Lane:   DETECTED (m={m:.3f}, b={b:.1f})")
        else:
            print(f"    Right Lane:   NOT DETECTED")

        pipeline_results[pipeline.fitter.method] = results
        print(f"    Total lines detected: {len(result['line_params'])}")

print(f"\n    Processing complete\n")


# Compute validation metrics
print("[Validation Metrics]")
print("-" * 80)
def compute_metrics(results):
    """Compute detection rates and average lines detected."""
    detection_rates = {
        'left': sum(1 for r in results if r['left_fit'] is not None) /␣
 ↪len(results),
        'right': sum(1 for r in results if r['right_fit'] is not None) /␣
 ↪len(results),
        'both': sum(1 for r in results if r['left_fit'] is not None and␣
 ↪r['right_fit'] is not None) / len(results)
    }

    avg_lines_detected = np.mean([len(r['line_params']) for r in results])

    print(f"   Total Test Images:            {len(results)}")
    print(f"   Left Lane Detection Rate:     {detection_rates['left']*100:.
 ↪1f}%")
    print(f"   Right Lane Detection Rate:    {detection_rates['right']*100:.
 ↪1f}%")
```

```
    print(f"   Both Lanes Detection Rate:    {detection_rates['both']*100:.
↪1f}%")
    print(f"   Average Lines per Image:     {avg_lines_detected:.1f}")
    print(f"    Metrics computed\n")

for method, results in pipeline_results.items():
    print("-" * 80)
    print(f"Method: {method.upper()}")
    print("-" * 80)
    compute_metrics(results)
    print("-" * 80)
```

```
[6-7] VALIDATION & INFERENCE
--------------------------------------------------------------------------------

   Processing Image 1/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.736, b=569.9)
     Right Lane:   DETECTED (m=0.782, b=142.7)
     Total lines detected: 99

   Processing Image 2/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.557, b=493.2)
     Right Lane:   DETECTED (m=0.712, b=262.7)
     Total lines detected: 23

   Processing Image 3/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.735, b=569.1)
     Right Lane:   DETECTED (m=0.821, b=45.2)
     Total lines detected: 155

   Processing Image 4/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
```

```
     Left Lane:    DETECTED (m=-1.014, b=568.9)
     Right Lane:   DETECTED (m=0.556, b=191.1)
     Total lines detected: 11

   Processing Image 5/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.959, b=675.1)
     Right Lane:   DETECTED (m=0.701, b=111.8)
     Total lines detected: 78

   Processing Image 6/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.422, b=476.5)
     Right Lane:   DETECTED (m=0.566, b=145.1)
     Total lines detected: 38

   Processing Image 7/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.559, b=477.5)
     Right Lane:   DETECTED (m=0.463, b=168.3)
     Total lines detected: 11

   Processing Image 8/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    NOT DETECTED
     Right Lane:   DETECTED (m=0.740, b=95.3)
     Total lines detected: 6

   Processing Image 9/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.718, b=515.9)
     Right Lane:   DETECTED (m=1.645, b=-144.5)
     Total lines detected: 43
```

```
   Processing Image 10/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.575, b=486.2)
     Right Lane:   DETECTED (m=0.580, b=174.5)
     Total lines detected: 130

   Processing Image 11/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-3.422, b=1288.4)
     Right Lane:   DETECTED (m=0.385, b=279.1)
     Total lines detected: 16

   Processing Image 12/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.383, b=517.2)
     Right Lane:   DETECTED (m=0.513, b=239.5)
     Total lines detected: 9

   Processing Image 13/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.490, b=588.0)
     Right Lane:   DETECTED (m=0.456, b=169.0)
     Total lines detected: 24

   Processing Image 14/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.664, b=490.3)
     Right Lane:   DETECTED (m=0.570, b=166.0)
     Total lines detected: 32

   Processing Image 15/20
[Pipeline] Starting image processing…
```

```
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-0.559, b=517.4)
     Right Lane:   DETECTED (m=0.594, b=116.4)
     Total lines detected: 111

   Processing Image 16/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    NOT DETECTED
     Right Lane:   NOT DETECTED
     Total lines detected: 0

   Processing Image 17/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    DETECTED (m=-1.064, b=636.0)
     Right Lane:   DETECTED (m=0.563, b=162.0)
     Total lines detected: 67

   Processing Image 18/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    NOT DETECTED
     Right Lane:   NOT DETECTED
     Total lines detected: 0

   Processing Image 19/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
     Left Lane:    NOT DETECTED
     Right Lane:   DETECTED (m=0.368, b=228.1)
     Total lines detected: 6

   Processing Image 20/20
[Pipeline] Starting image processing…
[Pipeline] Line fitting done.
    Image processed
result <class 'dict'>
```

```
    Left Lane:    DETECTED (m=-0.680, b=502.5)
    Right Lane:   DETECTED (m=0.856, b=-28.3)
    Total lines detected: 181

   Processing complete

[Validation Metrics]
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
Method: AVERAGING
--------------------------------------------------------------------------------
   Total Test Images:            20
   Left Lane Detection Rate:     80.0%
   Right Lane Detection Rate:    90.0%
   Both Lanes Detection Rate:    80.0%
   Average Lines per Image:     52.0
    Metrics computed

--------------------------------------------------------------------------------
```

[8]:
```python
def visualize_results(results, pipeline, indices):
    """Visualize results for test images."""
    n_results = len(results)

    fig, axes = plt.subplots(n_results, 4, figsize=(16, 4*n_results))
    if n_results == 1:
        axes = axes.reshape(1, -1)

    for idx, (result, img_idx) in enumerate(zip(results, indices)):
        # Original + Processed side-by-side
        try:
            orig = result['original']  # RGB HxWx3
            proc = result['processed']  # grayscale HxW
            # Ensure proc is uint8 and 2D
            proc = np.squeeze(proc).astype(np.uint8)
            # Convert to 3-channel RGB
            proc_rgb = cv2.cvtColor(proc, cv2.COLOR_GRAY2RGB)
            # Resize proc_rgb if necessary to match original
            if proc_rgb.shape[:2] != orig.shape[:2]:
                proc_rgb = cv2.resize(proc_rgb, (orig.shape[1], orig.shape[0]))
            side_by_side = np.hstack([orig, proc_rgb])
            axes[idx, 0].imshow(side_by_side)
            axes[idx, 0].set_title('Original | Processed')
            axes[idx, 0].axis('off')
        except Exception:
            # Fallback: show original only
            axes[idx, 0].imshow(result['original'])
```

```
            axes[idx, 0].set_title('Original')
            axes[idx, 0].axis('off')

        # Processed + Canny edges
        axes[idx, 1].imshow(result['edges'], cmap='gray')
        axes[idx, 1].set_title('Canny Edges')
        axes[idx, 1].axis('off')

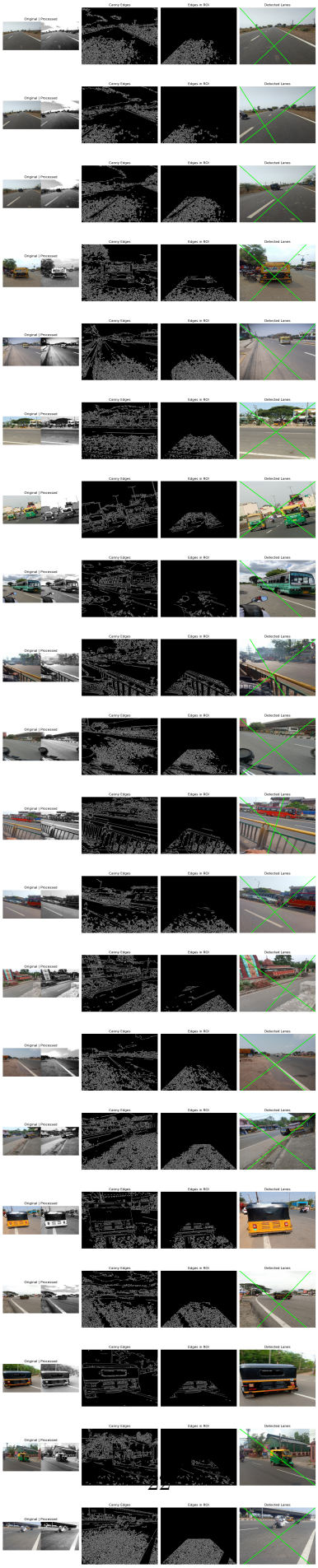
        # Masked edges (ROI applied)
        axes[idx, 2].imshow(result['masked_edges'], cmap='gray')
        axes[idx, 2].set_title('Edges in ROI')
        axes[idx, 2].axis('off')

        # Final detection
        final_image = pipeline.draw_lanes(result['original'],
                                          result['left_fit'],
                                          result['right_fit'])
        axes[idx, 3].imshow(final_image)
        axes[idx, 3].set_title('Detected Lanes')
        axes[idx, 3].axis('off')

    plt.tight_layout()
    plt.savefig('lane_detection_results.png', dpi=150, bbox_inches='tight')
    print("   Saved visualization: lane_detection_results.png")
    plt.show()
# Visualize results
for method, results in pipeline_results.items():
    print("-" * 80)
    print(f"\nVisualizing results for method: {method.upper()}")
    visualize_results(results, pipelines[0], test_indices)
# ================================================================================
# END OF CODE
# ================================================================================
```

```
--------------------------------------------------------------------------------

Visualizing results for method: AVERAGING
   Saved visualization: lane_detection_results.png
```


Original | Processed
Canny Edges
Edges in ROI
Detected Lanes